

Week 3 Assignment Report

Reinforcement Learning

WiDS Kalman Filtered Trend Trader – Assignment 3

Your Name

January 6, 2026

Contents

1	Introduction	3
2	Task 1: Environment Setup	3
2.1	Objective	3
2.2	Implementation	3
2.3	Results	3
2.4	Observations	4
3	Task 2: Q-Learning on FrozenLake	4
3.1	Environment Description	4
3.2	Algorithm: Tabular Q-Learning	4
3.3	Hyperparameters	4
3.4	Implementation Details	5
3.5	Results	5
3.6	Why the Success Rate is Not 100%	5
3.7	Challenges Faced	6
4	Task 3: Deep Q-Learning on MountainCar	6
4.1	Environment Description	6
4.2	Why Tabular Q-Learning Does Not Work Well	6
4.3	Deep Q-Network (DQN) Idea	6
4.4	Neural Network Architecture	7
4.5	Hyperparameters	7
4.6	Reward Shaping	7
4.7	Training Behaviour and Observations	7
4.8	Challenges Faced	8
5	Comparison and Analysis	8
5.1	When to Use Each Approach	8
5.2	Key Insights	8
6	Conclusion	9

1 Introduction

In this assignment, I explored reinforcement learning (RL) using two different environments:

- **FrozenLake:** A small grid-world environment with a discrete state space. This is a good fit for *tabular Q-learning*.
- **MountainCar:** A classic control problem where the state is continuous (position and velocity). This needs a function approximator like a *Deep Q-Network (DQN)*.

The overall goal was to understand how different RL methods work, and why we choose different algorithms depending on the environment.

2 Task 1: Environment Setup

2.1 Objective

The goal of Task 1 was to confirm that the Gymnasium library was correctly installed and that I could create and interact with the **FrozenLake-v1** environment.

2.2 Implementation

I wrote a small verification script (`task1_environment_setup.py`) that:

1. Imports Gymnasium
2. Creates the FrozenLake environment
3. Resets the environment and prints the initial state
4. Takes a few random actions to check that the environment responds correctly
5. Prints basic environment specifications

2.3 Results

The environment setup worked successfully. The environment details are:

- **State space:** 16 discrete states (a 4×4 grid)
- **Action space:** 4 actions (Left, Down, Right, Up)
- **Initial state:** 0 (top-left cell, Start position)

2.4 Observations

FrozenLake is *stochastic* when `slippery=True`. This means the agent may not move exactly in the direction it chooses. This makes learning harder because the same action can lead to different outcomes.

3 Task 2: Q-Learning on FrozenLake

3.1 Environment Description

FrozenLake is a 4×4 grid world:

- **S (Start):** top-left corner
- **G (Goal):** bottom-right corner
- **H (Holes):** falling into a hole ends the episode with reward 0
- **F (Frozen):** safe tiles
- **Reward:** +1 only when reaching the goal, 0 otherwise

Because the ice is slippery, the environment is not fully predictable.

3.2 Algorithm: Tabular Q-Learning

Q-learning is a model-free algorithm that learns an action-value function $Q(s, a)$, meaning the expected reward if we take action a in state s and then act optimally.

The update rule used is:

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right]$$

where:

- α is the learning rate
- γ is the discount factor
- r is the reward after taking action a
- s' is the next state

3.3 Hyperparameters

After experimenting with multiple values, I used the following settings:

Parameter	Value	Simple Reason
Learning rate (α)	0.1	Learns steadily without oscillating too much
Discount factor (γ)	0.95	Values future rewards but not too aggressively
Initial epsilon (ϵ)	1.0	Full exploration at the start
Minimum epsilon	0.01	Keeps some exploration forever
Epsilon decay	0.995	Slow decay helps in a stochastic world
Episodes	10,000	Enough time to learn a good policy
Max steps/episode	100	Grid is small, so 100 steps is plenty

3.4 Implementation Details

The main pieces of implementation were:

- Initialize the Q-table with all zeros
- Use an **epsilon-greedy** policy:
 - With probability ϵ : choose a random action (explore)
 - With probability $1 - \epsilon$: choose the best known action (exploit)
- Update the Q-table after every step using the Q-learning update rule
- Decay ϵ after every episode to slowly reduce exploration

3.5 Results

Final performance (tested on 100 evaluation episodes):

- **Success rate:** ~ 0.74 (74%)
- **Final training success rate (last 100 episodes):** ~ 0.72
- **Final epsilon:** 0.01

3.6 Why the Success Rate is Not 100%

Even if the agent learns the best possible policy, FrozenLake is slippery. This means:

- The agent can slip into a hole even after choosing the correct move.
- Randomness makes perfect performance impossible.

So 70–80% success is actually a strong result here.

3.7 Challenges Faced

Some practical challenges I observed:

- Rewards are sparse (only at the goal), so learning starts slow
- Stochastic transitions make learning noisy
- Hyperparameters matter a lot, so I had to tune them carefully

4 Task 3: Deep Q-Learning on MountainCar

4.1 Environment Description

MountainCar is a control task where the agent must move a car to a goal position (≥ 0.5). The problem is that the engine is too weak to drive straight to the goal. The agent must learn to swing back and forth to build momentum.

- **State space:** Continuous (position, velocity)
- **Action space:** Discrete (push left, no push, push right)
- **Reward:** -1 per time step until reaching the goal

4.2 Why Tabular Q-Learning Does Not Work Well

Tabular Q-learning struggles because:

- There are infinitely many states (continuous values)
- Discretizing either loses information (too coarse) or becomes huge (too fine)
- A table does not generalize between similar states

4.3 Deep Q-Network (DQN) Idea

DQN replaces the Q-table with a neural network that approximates $Q(s, a)$. Two important ideas make DQN stable:

- **Experience replay:** store transitions and train on random batches
- **Target network:** use a separate network to compute stable targets

4.4 Neural Network Architecture

The DQN architecture used was:

- Input: 2 values (position, velocity)
- Hidden layer 1: 128 units with ReLU
- Hidden layer 2: 128 units with ReLU
- Output: 3 values (Q-values for 3 actions)

4.5 Hyperparameters

Parameter	Value	Reason
Learning rate	0.001	Standard for Adam
Discount factor (γ)	0.99	Needs long-term planning
Initial epsilon	1.0	Full exploration at start
Minimum epsilon	0.01	Keeps some exploration
Epsilon decay	0.995	Slow decay helps delayed rewards
Batch size	64	Common training batch size
Replay buffer size	10,000	Stores diverse experience
Target update frequency	Every 10 episodes	Improves stability
Episodes	500	Enough for learning improvement

4.6 Reward Shaping

The default reward is very sparse (-1 every step). To help learning, I added reward shaping:

- $+0.1$ for moving right (toward the goal)
- $+100$ for reaching the goal

This does not change the goal of the task, but it helps the agent learn faster by giving more feedback.

4.7 Training Behaviour and Observations

The learning behaviour can be explained in phases:

- **Episodes 0–100:** Mostly random behaviour, little improvement
- **Episodes 100–300:** Begins learning the swing strategy, occasional successes
- **Episodes 300–500:** More consistent success, shorter episodes

4.8 Challenges Faced

- Delayed rewards make credit assignment difficult
- Learning looks slow initially, but improves after replay buffer fills
- Hyperparameters are sensitive, especially epsilon decay

5 Comparison and Analysis

5.1 When to Use Each Approach

Tabular Q-learning is good for:

- Small and discrete environments
- Easy debugging and interpretability

Tabular Q-learning is not good for:

- Continuous state spaces
- Large state spaces

DQN is good for:

- Continuous states
- Problems where generalization matters
- Bigger environments where tables are impractical

DQN is not good for:

- Very small problems (overkill)
- Cases where interpretability is extremely important

5.2 Key Insights

- Exploration is essential in both environments
- FrozenLake randomness prevents perfect performance
- MountainCar needs long-term planning, so learning takes time
- DQN works well because it generalizes over similar states
- Hyperparameter tuning is unavoidable in RL

6 Conclusion

This assignment gave practical experience with two important RL approaches:

- **Q-learning on FrozenLake:** Worked well and achieved around 74% success rate, which is strong given the stochastic nature of the environment.
- **DQN on MountainCar:** Demonstrated why deep RL is needed for continuous states and showed how replay buffers and target networks help stabilize learning.

Overall, the main takeaway is that algorithm choice depends heavily on the environment. Small discrete problems can be solved with simple tabular methods, but continuous or complex problems require function approximation like deep neural networks.